

F1R3Gaze: Implementation Specification

A browser whose only execution mechanism is f1r3lang on a native RSpace

Version 0.1 (draft for review), companion to the architecture proposal v0.2

Lucius Gregory Meredith

CEO, F1R3FLY.io, and Chief Science Officer, F1R3FLY Industries

28 September 2026

Abstract

The architecture proposal in `publications/f1r3gaze` settles *what* the browser is: Blitz as the document engine, the CampF1R3 executive as the only execution mechanism, a F1R3FLY shard as the server, Servo quarantined for legacy pages, and f1r3x-wasm as the reach tier. This document settles *how* to build it. It pins the code it builds on, records what a line-by-line review of that code found — including four places where the proposal assumes machinery that does not yet exist — and then specifies, to the level of types, wire forms and algorithms, the upstream changes to CampF1R3, the .knf container, the tab executive and its frame loop, the replay log, the DOM protocol, every capability’s protocol, the shard bridge and its light-client check, key-bound sessions, the graded resolver, the process model, devtools, the reach tier, the legacy tab, the projection gateway, and the conformance suite. It closes with work packages mapped onto the proposal’s five gates, and with the obligations and decisions that remain open. The working name F1R3Gaze follows the name of the directory in which the proposal lives.

Contents

1	Scope and conventions	3
1.1	What this document is	3
1.2	Pinned baselines	3
2	Findings from the code review	3
3	Workspace	5
4	Upstream changes to CampF1R3	5
4.1	U1: ground data (level K1G, feature ground)	6
4.2	U2: guards (level K2, feature guards)	6
4.3	U3: an observer-generic machine	7
4.4	U4: apertures	7
4.5	U5: the resolver seam and the Graded profile	7
4.6	U6: a keyed minter	8
4.7	U7: authenticated attach in f1r3x-serve	8

5	The .knf container and how pages load	8
5.1	Format	8
5.2	Identity	9
5.3	In HTML	9
5.4	Load sequence	9
6	The tab executive	9
6.1	Structure	9
6.2	The frame	10
6.3	The replay log	10
7	The DOM protocol	11
7.1	Conventions	11
7.2	Verbs	12
7.3	Events	12
8	Capabilities and the broker	13
8.1	Grants	13
8.2	Protocols	13
8.3	The page meter	14
9	The shard bridge	14
9.1	Components	14
9.2	Addresses	14
9.3	Site manifest	14
9.4	Resolution	15
9.5	The assurance ladder in code	15
9.6	Page verbs	15
9.7	Blobs	15
9.8	Sessions	16
10	The graded resolver	16
11	Process model	16
12	Devtools, reach, legacy and gateway	17
13	Testing and conformance	17
14	Work packages and schedule	18
15	Obligations and decisions	19
15.1	Proposal decisions this specification settles, subject to review	20
15.2	Still open	20
A	The lamp, in K1G	20

1 Scope and conventions

1.1 What this document is

The proposal (v0.2) is normative for architecture; this specification is normative for implementation. Where the two disagree, this document says so explicitly and gives the reason, which in every case is something found in the code. The key words MUST, MUST NOT, SHOULD and MAY have their usual meaning.

1.2 Pinned baselines

Everything below was read at the commits in Table 1. Work packages MUST re-pin at the start of each phase and re-run the findings of Section 2 against the new pins.

Repository	Commit (date)	Role here
F1R3FLY-io/campf1r3	b46e16c (18 Sep 2026)	the executive: 11 crates, $\approx$ 12,600 lines, 170 tests, no external dependencies
F1R3FLY-io/f1r3node-rust	fce422a (18 Sep 2026), master	the shard: HTTP API, /ws/events, VersionedRegistry.rho, rho-pure-eval, cost accounting
F1R3FLY-io/publications	HEAD, 28 Sep 2026	f1r3gaze (proposal), campf1r3 (store, kernel, byte-code specs), fuzzyware (graded where-clauses)
F1R3FLY-io/embers	HEAD, 28 Sep 2026	prepare/sign/send deploy pattern; firefly-client crate
F1R3FLY-io/F1R3Drive	0f3849a	chunked on-chain file layout; write-back cache with bulk deploys
F1R3FLY-io/F1R3Sky	57dcae5	client-held keys through @f1r3fly-io/embers-client-sdk
DioxusLabs/blitz	674d7d2 (28 Sep 2026), 0.3.0-beta.2	document engine; Rust 1.91, edition 2024

Table 1: Baselines. The node pins nightly-2026-02-09; CampF1R3 targets Rust 1.75 and edition 2021; Blitz requires 1.91 and edition 2024.

Requirement 1 (Toolchain). The F1R3Gaze workspace MUST build on stable Rust $\geq$ 1.91, edition 2024. CampF1R3 crates MUST be consumed unmodified as git dependencies at a pinned revision (edition-2021 crates compile inside an edition-2024 workspace), and every change to them MUST land upstream first (Section 4). No crate in the page path MAY require nightly.

2 Findings from the code review

Each finding states what the proposal assumes, what the code does, and what this specification does about it.

Finding 1 (No ground data in the kernel). The proposal’s examples send strings (doc! ("query1", "#lamp", *found)). The K1ndl1ng specification lists “ground literals of every sort, collections, methods” as explicitly out of scope, and k1ndl1ng-norm’s Node has no literal constructor. *Response*: a ground-data sort, K1G, specified in §4.1 and scheduled into P0.

Finding 2 (Guards are parsed and discarded). At K2 the parser recognises where and then skips tokens to the closing parenthesis; nothing is recorded and nothing evaluates. The node's `rho-pure-eval` cannot be reused in the client: it depends on `models`, which depends on `rspace++` (`heed`, `tokio`) and `tonic`. *Response*: a portable evaluator, `k1nd11ng-eval` (§4.2), differentially tested against `rho-pure-eval`.

Finding 3 (No host channels, and no way to meter). `b3110ws::Machine` owns an `RSpace` fixed to `NoObserver`, so the page meter cannot be installed; and a send on a grounded name can only be seen by the host by polling the store. *Response*: a `Machine` generic over its observer, and *apertures* (§4.4).

Finding 4 (No oracle seam). The proposal speaks of an oracle ω to which a Graded profile attaches. In the code, choice is first-match inside `Space::try_candidate`, and `Profile` is an enum whose `Free` variant is currently treated as `Deterministic`. *Response*: a `Resolver` trait over enumerated candidates (§4.5).

Finding 5 (Mixed persistence in joins is well defined here). `b3110ws::receipt_parts` makes a continuation persistent if *any* bind is persistent, while data persistence stays per datum. The proposal's lamp, for `(_ <= clicks & _ <- off)`, therefore means “a persistent handler that consumes one click and the off token”, which is what was intended. The example stands; Appendix A gives it in K1G form.

Finding 6 (Blitz dispatch is synchronous). `blitz_dom::EventHandler::handle_event` receives an `EventState` and must decide `prevent_default` and `stop_propagation` before it returns. The proposal's “apply the default after a deadline” cannot be implemented on it. *Response*: static listener options, plus an opt-in synchronous drain bounded by a count of communications (§7.3).

Finding 7 (Blitz is one process, and already hosts a script engine). `blitz-shell` drives a `Box<dyn Document>` in the window's process; there is no renderer/browser split. `anyrender_serialize` exists and makes a display-list transport feasible. Blitz now also ships `blitz-vibey-script`, a Boa-backed `ScriptDocument`; its shape (wrap `BaseDocument`, implement `Document::handle_ui_event` and `Document::poll`) is the template for `RhoDocument`. *Response*: one OS process with a worker thread per tab through P3; process separation in P4 (§11). `blitz-vibey-script` MUST NOT be linked into any binary that hosts `f1r3lang` pages.

Finding 8 (Node API, registry and costs). The node serves more than the proposal lists: `/api/explore-deploy`, `/api/explore-deploy-by-block-hash`, `/api/data-at-name-by-block-hash`, `/api/registry/{uri}` with a `block_hash` query parameter, `/api/estimate-cost`, `/api/prepare-deploy`, `/api/deploy-finalization-status/{sig}`, and `/ws/events` carrying `block-added`, `block-created`, `block-finalised` and `deploy lifecycle` events. There is no inclusion-proof endpoint. In `VersionedRegistry.rho` the header records that Step 6 (the public `rho:registry:1.0.0` entry point) and Step 7 (notify firing) were pending; the notify list is captured but “ignored for now”. Storage cost is computed from prost-encoded sizes of `Pars`. *Response*: freshness by re-resolution on `block-finalised` until Step 7 lands; the client meter is a local budget, and deploy costs are quoted from `/api/estimate-cost` (§8.3).

Finding 9 (Session names in `f1r3x-serve` are bearer tokens). A client attaches at a path of the form `ws://host/s/<session-name>`, choosing the name itself, and “an unknown name creates the session”. Whoever learns the name owns the session. *Response*: authenticated attach (§4.7).

Finding 10 (Existing F1R3FLY clients already do half the shard bridge). Embers exposes each mutation as `/prepare` (the server renders a `f1r3lang` template and returns an unsigned contract under a JWT) and `/send` (the client returns it signed); `F1R3Sky` holds the private key in the client through

the Embers SDK. Embers’ `firefly-client` crate already signs `DeployData` with `secp256k1`, submits over gRPC and HTTP, and consumes `/ws/events`. `F1R3Drive` stores a file as a metadata map with a first chunk inline and further chunks on sub-channels, deployed in bulk from a write-back cache. *Response*: the shard bridge depends on `firefly-client` (§9); `F1R3Drive`’s layout is the on-chain fallback for blobs (§9.7).

Finding 11 (One missing adapter). The proposal cites a `campf1r3-ispac` adapter for differential replay against `rspace++`. It is not in the pinned repository. Obligation 1 carries it.

3 Workspace

A new repository, `F1R3FLY-io/f1r3gaze`, Apache-2.0, holds everything that is not an upstream change. Crates, in dependency order:

Crate	Target	Contents
<code>gaze-ground</code>	all	Ground values, conversion to and from K1G normal forms, the reply-tuple conventions of §7.1
<code>gaze-knf</code>	all	the <code>.knf</code> container: reader, writer, manifest, integrity strings
<code>gaze-exec</code>	all	<code>TabExec</code> : executive, apertures, meter, resolver, recorder, frame loop
<code>gaze-graded</code>	all	semirings, resolvers, deterministic PRNG, fairness, ceiling check
<code>gaze-dom-core</code>	all	the DOM protocol engine over a <code>DomBackend</code> trait; name table; attenuation; event routing
<code>gaze-dom-blitz</code>	native	<code>DomBackend</code> over <code>blitz-dom</code> ; <code>RhoDocument</code> ; <code>RhoEventHandler</code>
<code>gaze-broker</code>	native	manifests to grants, policy, prompts, routing table, revocation
<code>gaze-net</code>	native	<code>blitz_traits::net::NetProvider</code> with hash verification; the net capability
<code>gaze-store</code>	native	per-origin key-value store behind the store capability
<code>gaze-shard</code>	native	shard bridge: keystore, deploys, registry, reads, proofs, sessions, watches
<code>gaze-blob</code>	native	<code>BlobSource</code> implementations and the content cache
<code>gaze-shell</code>	native	the application: windows, tabs, chrome, prompts, settings
<code>gaze-devtools</code>	all	debug capability; the devtools page is itself <code>f1r3lang</code>
<code>gaze-reach</code>	wasm32	<code>cdylib</code> bundling the executive and <code>gaze-dom-core</code> , plus <code>gaze-reach.js</code>
<code>gaze-legacy</code>	native	P4: Servo in its own process
<code>gaze-gateway</code>	native	P4: headless projection server
<code>gaze-conformance</code>	all	the protocol and determinism suites of §13
<code>f1r3c</code>	native	toolchain CLI: surface <code>f1r3lang</code> or kernel text to <code>.knf</code> ; site publishing

Requirement 2 (Portable core). `gaze-ground`, `gaze-knf`, `gaze-exec`, `gaze-graded`, `gaze-dom-core` and `gaze-devtools` MUST build for `wasm32-unknown-unknown`, MUST carry `#![forbid(unsafe_code)]`, and MUST NOT depend on `std::time`, `threads`, `sockets`, floating point on any path that affects output, or any crate outside the Rust standard library and `CampF1R3`.

This is the rule that makes the native browser and the reach tier one implementation: the protocol engine and the executive are the same code in both, and only the `DomBackend` differs.

4 Upstream changes to CampF1R3

Every change here is application-agnostic, so it respects the constraint recorded when `CampF1R3` was split from `Rhobots-in-Space`: nothing in that repository may know that a browser exists. Each

change is behind a cargo feature, off by default, so the Rhobots path and the 170 existing tests are unaffected.

4.1 U1: ground data (level K1G, feature ground)

Sort. A new process constructor carries ground values:

```
pub enum Node { /* existing variants */, Ground(Lit) }
pub enum Lit {
  Bool(bool), Int(i64), Str(Box<str>), Bytes(Box<[u8]>),
  List(Vec<Norm>), Tuple(Vec<Norm>), Map(Vec<(Norm, Norm)>),
}
```

Collection elements are arbitrary normal forms, so names (as **x*) may sit inside lists, tuples and maps. There are no floats: every consumer in this specification needs bit-identical results across targets.

Encoding. Tags continue the table of the K1nd11ng specification in the unused range between Wild (0x08) and Quote (0x10):

Tag	Constructor	Payload
0x09	Bool	one byte, 0 or 1
0x0A	Int	zigzag LEB128 i64
0x0B	Str	LEB128 length, UTF-8 bytes (no normalisation)
0x0C	Bytes	LEB128 length, bytes
0x0D	List	$n, \llbracket e_1 \rrbracket \cdots \llbracket e_n \rrbracket$, source order
0x0E	Tuple	$n, \llbracket e_1 \rrbracket \cdots \llbracket e_n \rrbracket$, source order
0x0F	Map	n , then pairs sorted by $\llbracket k \rrbracket$ bitwise; duplicate keys are a normalisation error

Requirement *order key is the encoding itself* of the K1nd11ng specification is preserved. Whether bitwise order on these tags agrees with the node’s models Score order for the corresponding Exprs is added to that specification’s sort-agreement obligation; until it is discharged, a normal form containing ground data MUST NOT be claimed equal to a node term by hash.

Surface syntax. Rholang’s literal syntax, as rholang-tree-sitter fixes it: true, false, decimal integers with optional sign, double-quoted strings with rholang’s escapes, [...], (a, b, ...) with at least two elements, {k: v, ...}. Bytes have no surface form at K1G; they occur only in .knf encodings and in data injected by a host, and the printer shows them as 0x... Methods remain out of scope.

Matching. In k1nd11ng-campf1r3::spatial, a ground pattern matches a ground datum iff their encodings are equal; collection patterns match elementwise, with FreeVar, Wild and Eval(Name::Free) binding inside them as they already do inside processes. Map patterns match exactly (no row variables) in v1.

4.2 U2: guards (level K2, feature guards)

A new crate, k1nd11ng-eval, pure and portable, evaluates the guard subset: ground literals, bound variables, == != < <= > >= on Int and on Str (bitwise), + - * / % on Int with checked

arithmetic (overflow and division by zero make the guard evaluate to an error, which is *not enabled*), ++ on Str, Bytes and List, and, or, not, and two graded scalars used only in graded pages: w(n, d) and cost(n) (§10). The parser records the guard as a tree instead of skipping it; the normal form stores it on the Receive node under a new tag 0x21 (Guard), after the binds and before the body.

Requirement 3 (Guard evaluation site). Guards MUST be evaluated in `Matcher::check_commit(k, bound)`, which is where the store already asks, once per candidate. For crisp pages the result is a Boolean; for graded pages it is routed to the resolver (U5).

4.3 U3: an observer-generic machine

```
pub struct Machine<B: Backend = Direct, M: NameMinter = Sequential,
    O: Observer<Chan, BindPat, Data, Cont> = NoObserver> { .. }
impl<B, M, O> Machine<B, M, O> {
    pub fn with_observer(cfg: Config, backend: B, minter: M, observer: O) -> Self;
}
```

`k1ndl1ng-campf1r3::rspace` gains `rspace_with(cfg, observer)`. `f1r3x::Executive` keeps its signature.

4.4 U4: apertures

An aperture is a channel owned by the host. The page can send on it; the store never sees those sends.

```
pub struct ApertureId(pub u32);
pub struct Outbound { pub aperture: ApertureId, pub args: Vec<Norm>, pub seq: u64 }

impl<B, M, O> Machine<B, M, O> {
    /// Sends on 'chan' go to the outbox instead of the store.
    pub fn bind_aperture(&mut self, chan: Name, id: ApertureId);
    pub fn unbind_aperture(&mut self, id: ApertureId);
    pub fn take_outbound(&mut self) -> Vec<Outbound>;
    /// Host-originated send: enqueued as a 'Send' job at the tail of the queue.
    pub fn inject(&mut self, chan: Name, args: Vec<Norm>);
}
```

- A Send job whose channel is bound to an aperture is appended to the outbox with a monotone seq, emits `Step::Outbound`, and calls a new `Observer::observe_aperture` hook, which may abort like any other.
- A persistent send on an aperture is delivered once and emits a diagnostic.
- A receipt on an aperture channel is stored and is inert: the host never produces on apertures, it injects on names the page gave it.
- `inject` is the only host write path. It does not bypass the store, the observer or the resolver.

4.5 U5: the resolver seam and the Graded profile

`Profile` gains `Graded`. In that profile `try_candidate` does not return the first admissible candidate; it enumerates up to `Config::max_candidates` (default 256) admissible candidates in the existing deterministic order, asks the matcher for each one's value, and asks the resolver to choose.

```

pub trait Matcher<P, A, K> {
  fn get(&self, p: &P, a: &A) -> Option<A>;
  fn check_commit(&self, k: &K, bound: &[A]) -> bool { true }
  /// Graded profile only: the clause's value, opaque to the store.
  fn weigh(&self, k: &K, bound: &[A]) -> Weight { Weight::ONE }
}

pub struct Weight(pub u128);          // semiring-specific packing
pub trait Resolver: Send + Sync + 'static {
  /// 'ws' in candidate order; None means no candidate is enabled.
  fn select(&self, site: Hash32, ws: &[Weight]) -> Option<usize>;
}

```

The log gains `Event::Select { site, n, chosen }`. In Replay the selection is read back from the log rather than recomputed, so sampled runs replay exactly. Truncation at `max_candidates` emits a diagnostic and is recorded.

Remark 1 (Local versus contention-set resolution). The store resolves at a cut: the candidates are those that use the arriving record. The quantum paper normalises over the *contention component*, the connected component of the conflict relation over the whole term. For idempotent semirings (Boolean, Viterbi, tropical) the cut-local argmax is what a page asks for. For sampled $\mathbb{R}_{\geq 0}$ pages it is an approximation of the paper's semantics, and the specification says so rather than hiding it. An exact *round mode*, which collects the enabled sites of an admitted round, builds the conflict graph on shared occurrences and resolves per component, is Obligation 2.

4.6 U6: a keyed minter

Sequential is a pure function of binder path and counter, which is right for Rhobots and wrong where names might be exported. Keyed `{ seed: [u8; 32], counter: u64 }` mints `blake2b_256("k1ndl1ng/keyed/" || seed || counter || path)`. The browser seeds it from the OS at tab creation and records the seed in the replay log.

4.7 U7: authenticated attach in f1r3x-serve

ready gains a 32-byte nonce. A client attaches with `auth <pubkey-hex> <sig-hex>` over the nonce and the session path, and the session is keyed by the public key; the path segment becomes a label. An unauthenticated attach is refused unless the server is started with `--open`, which the browser never uses. The Rhobots client is updated in the same change.

5 The .knf container and how pages load

5.1 Format

Field	Contents
magic	F1R3KNF\0 (8 bytes)
version	u16, 1
level	u8: 0 K0, 1 K1, 2 K1G, 3 K2 (implies K1G)
sections	repeated (tag u8, len LEB128, bytes), tags ascending, each at most once
0x01 manifest	a K1G Map (below), encoded
0x02 body	the <i>ungrounded</i> normal-form encoding
0x03 source map	optional; spans for diagnostics and devtools

The manifest is:

```
{ "imports": [ ("doc", "rho:gaze:doc"), ("net", "rho:gaze:net"), ... ],
  "semiring": "boolean" | "viterbi" | "tropical" | "prob",
  "fair":      false,
  "ceiling":   "crisp" | "idempotent" | "sampled",
  "budget":    { "frame": 20000, "sync": 512 } }
```

imports lists every free identifier of the body in level order with the capability it requests. An identifier the manifest does not list is a load error.

5.2 Identity

Requirement 4 (Two hashes). The *program hash* is `k1nd1ng-norm`'s BLAKE2b-256 of the body, exactly as the executive computes it; it keys the parse and plan caches. The *grant hash* is BLAKE2b-256 of `program hash || enc(manifest)`; it keys stored grants, because the manifest, not the body, states the authority requested. The HTML integrity attribute MUST carry the program hash as `blake2b-256:<64 hex>`.

5.3 In HTML

```
<script type="application/f1r3lang" src="app.knf"
      integrity="blake2b-256:9f2c...e1"></script>
```

Inline kernel text is accepted with `level`, `imports` and `semiring` attributes standing in for the manifest; the loader parses it at the declared level and builds the manifest from the attributes. A document may carry several `f1r3lang` scripts; each is a separate deploy into the same tab executive, in document order, sharing grants by identifier. Scripts of any other type are never executed; if a document has JavaScript scripts and no `f1r3lang` script, the shell offers to reopen it in a legacy tab (§12).

5.4 Load sequence

1. Blitz parses the document; `RhoDocument` collects `f1r3lang` scripts.
2. Each `src` is fetched through `gaze-net`; its program hash is recomputed and compared with `integrity`. A mismatch is fatal to the page.
3. The broker turns the manifest into grants (§8), prompting where policy requires. The tab shows the grants before anything runs.
4. For each import the broker returns a 32-byte key; the executive binds an aperture on `Name::Unforgeable(key)` and grounds the body with `ground_free(keys)`. Denied imports are grounded to fresh keys that are bound to nothing: dead channels.
5. The grounded term is spawned; the first frame runs.

6 The tab executive

6.1 Structure

```
pub struct TabExec {
  machine: Machine<Direct, Keyed, PageMeter>,
```

```

apertures: BTreeMap<ApertureId, Service>, // doc, net, store, ...
inbox:     Vec<Injection>,                // filled by services between frames
recorder:  Recorder,                      // the replay log, section 6.3
dom:       gaze_dom_core::Engine,
frame:     u64,
}

```

6.2 The frame

`TabExec::frame(now_ms)` is the whole driver. It is a pure function of the executive state, the inbox and `now_ms`.

1. **Order the inbox** by (source class, arrival sequence) with classes in the fixed order: DOM replies, timers, input events, network, store, shard. Record `Frame{n, now_ms}` and then each `Inject`.
2. **Inject** each entry with `Machine::inject`.
3. **Run** `machine.run(budget)` where `budget` is the manifest's frame budget in steps, reduced by the meter's remaining allowance.
4. **Drain** `take_outbound()` in seq order. DOM messages go to `gaze-dom-core`, which answers reads from the snapshot taken at the start of the frame and appends writes to the frame's commit batch; everything else goes to its service, which will answer by adding to the inbox of a later frame.
5. **Commit**: the batch is handed to the `DomBackend` and applied with one `DocumentMutator` and one `flush()`. Replies the DOM engine produced (names of created nodes, read results) enter the inbox for frame $n + 1$.

Requirement 5 (Frame atomicity, restated). Writes produced in frame n MUST become visible to layout together, after step 5. Reads in frame n MUST observe the document as committed at the end of frame $n - 1$, and their replies MUST arrive in frame $n + 1$. A page therefore never observes a half-applied frame, and a read is never answered from state that another read in the same frame could not see.

Remark 2 (Why not `Session::poll`). The proposal drives the executive through `f1r3x-host::Session::poll`. A `Session` exists to serialise rounds and scenes for a client across a boundary; in the page process the renderer is in-process and needs the outbox, not scenes. `TabExec` therefore drives the machine directly and keeps the host's admission rule for the debugger's lockstep mode. `Session` is used where it belongs: when devtools or a Rhobots headset attaches to a tab (§12).

6.3 The replay log

A `.gzlog` file is a header and a record stream.

Header

program hashes of all scripts, grant hash, grants by capability kind (never keys or secrets), minter seed, profile, semiring, executive and bridge build identifiers.

Records

`Frame{n, now_ms}`; `Inject{class, aperture_or_name, args}` with arguments as K1G encodings; `Sync{listener, event}` for synchronous drains; `Select{site, n, chosen}` copied from the store log; `Commit{n, batch_hash}`; `Budget{n, exhausted}`.

Replay loads the header, rebuilds the tab with the same seed and with services replaced by the log, and re-injects each record at its frame. The observable result — the committed document at every frame, compared by `batch_hash` — MUST be identical on x86_64, aarch64 and wasm32.

7 The DOM protocol

This section discharges the proposal's Obligation on the DOM protocol. It is implemented once, in `gaze-dom-core`, over:

```
pub trait DomBackend {
    type Node: Copy + Ord;
    fn query_in(&self, root: Self::Node, sel: &str, all: bool) -> Result<Vec<Self::Node>, SelErr>;
    fn read(&self, n: Self::Node, what: Read) -> ReadResult;
    fn apply(&mut self, batch: &[Write<Self::Node>]) -> Vec<Created<Self::Node>>;
    fn is_descendant(&self, n: Self::Node, of: Self::Node) -> bool;
}
```

`gaze-dom-blitz` implements it with `BaseDocument::query_selector_all_in`, `DocumentMutator` and `NodeId`; `gaze-reach` implements it over the host browser's DOM.

7.1 Conventions

- Every request is a send on an element name: `E!(verb, args..., ret)` where `verb` is a `Str` and `ret` is a name, or `E!(verb, args...)` for writes that need no acknowledgement.
- Every reply is one tuple on `ret`: `("ok", v)` or `("err", code, detail)`. Receivers match with patterns such as `for (@("ok", *el) <- r)`.
- Element names are unforgeable names minted by the engine, one per live node, stable for the node's life: asking twice for the same node yields the same name, so name equality is node equality.
- Error codes: "attenuated" (outside the holder's subtree), "detached" (node removed), "selector", "type" (wrong argument sort), "verb", "quota", "revoked".

Principle 1 (Attenuation is the tree, enforced). Every verb that returns or addresses a node MUST check `is_descendant(target, holder_root)` where `holder_root` is the node of the name the request was sent on. `parent` and `closest` never return a node above the holder.

7.2 Verbs

Request	Meaning and reply
<code>query1(sel, ret)</code>	first match in the subtree; ("ok", *E) or ("err", "none", sel)
<code>query(sel, ret)</code>	all matches; ("ok", [*E1, ...])
<code>children(ret),</code> <code>parent(ret), closest(sel,</code> <code>ret)</code>	navigation, attenuated
<code>attr(k, ret), setAttr(k,</code> <code>v), removeAttr(k)</code>	attributes; attr replies ("ok", v) or ("ok", Nil)
<code>text(ret), setText(s)</code>	text content
<code>value(ret), setValue(s)</code>	form-control values
<code>classAdd(c),</code> <code>classRemove(c),</code> <code>classToggle(c)</code>	class list
<code>style(k, v), unstyle(k)</code>	inline style properties
<code>append(frag, ret),</code> <code>prepend, before, after,</code> <code>replaceWith</code>	insert a fragment; replies ("ok", {ref: *E}) for every ("ref", k) marker
<code>setHTML(html)</code>	parse with <code>html5ever</code> and replace children; scripts in the fragment are inert
<code>remove()</code>	detach; the name dies
<code>rect(ret)</code>	layout box as (x, y, w, h) in integer CSS pixels, from the last committed layout
<code>focus(),</code> <code>blur(),</code> <code>scrollIntoView()</code>	focus and scroll
<code>listen(type, ch, opts,</code> <code>ret)</code>	subscribe (§7.3); replies ("ok", *sub); sub! ("cancel")

A fragment is either a `Str` of HTML or a structured term, which avoids string parsing and is the preferred form:

```
("el", "li", {"class": "todo"}, [ ("text", "buy_milk"), ("ref", "check") ])
```

The document root name `doc` additionally answers `title`, `setTitle`, `head`, `body` and `location`. Navigation is the `nav` capability, not a document verb.

7.3 Events

`listen(type, ch, opts, ret)` with `opts` a map of Booleans: `capture`, `once`, `prevent`, `stop`, `decide`. Each occurrence is injected as one datum on `ch`:

```
( type, { "target": *E, "x": 120, "y": 44, "button": 0, "key": "Enter",
          "mods": ["shift"], "value": "...", "reply": *R } )
```

Fields absent for a type are absent from the map. `target` is always inside the listener's subtree, since the listener was installed on a name for that subtree.

`RhoEventHandler` implements `blitz_dom::EventHandler`. For each `DomEvent` it walks Blitz's chain, capture phase then bubble phase, and for each node with listeners of that type:

1. applies static options: `prevent` calls `EventState::prevent_default`, `stop` calls `stop_propagation` after this node;
2. if `decide` is false (the default), queues the datum for the next frame;
3. if `decide` is true, injects the datum now and runs the tab executive synchronously for at most the manifest's sync budget in steps, then reads whatever "prevent" and "stop" messages

arrived on R. The drain is recorded as Sync. If the budget expires, the default action proceeds. The budget is in steps, not milliseconds, so the decision is deterministic and replayable. Text editing, caret movement and IME composition are Blitz's; the page sees input events with value.

8 Capabilities and the broker

8.1 Grants

The broker maps each manifest import to a grant. A grant is a routing-table entry (`tab`, `ApertureId`) $\mapsto$ (`service`, `attenuation`, `quota`); the page holds only the aperture's unforgeable name.

Principle 2 (Revocation is a table update). The proposal revokes by killing a forwarder process. With apertures the forwarder is the routing entry: removing it makes every later send on the name answer ("err", "revoked"), and the page cannot tell a revoked name from a dead one.

Capability	Default policy	Attenuation
<code>rho:gaze:doc</code>	granted	the document root; components get subtrees by name passing
<code>rho:gaze:log</code>	granted	console of this tab
<code>rho:gaze:clock</code>	granted	frame ticks, timers, frame-quantised time
<code>rho:gaze:rand</code>	granted	OS entropy, recorded in the log
<code>rho:gaze:net</code>	granted, same site	origin allow-list; widened only by prompt
<code>rho:gaze:store</code>	granted	this origin's space; quota
<code>rho:gaze:nav</code>	granted	same-site navigation; cross-site opens a new tab
<code>rho:gaze:shard</code>	prompt	per verb class: read, explore, deploy, session

Grants are stored per (site identity, grant hash); a new manifest is a new request. The shell lists a tab's grants and open sessions, and any entry can be revoked.

8.2 Protocols

clock

("frames", `ch`) injects ("tick", `n`, `now_ms`) each frame until ("stop", `ch`); ("after", `ms`, `ch`) fires once; ("now", `ret`) replies with the current frame's time.

log

(`level`, `v`) with level "debug", "info", "warn" or "error"; `v` is printed with `show_pretty`.

rand

("u64", `ret`), ("bytes", `n`, `ret`).

net

("fetch", {"url", "method", "headers", "body", "integrity"}, `ret`) replies ("ok", {"status", "headers", "body", "digest"}). With integrity set, the body is verified before delivery and a mismatch is ("err", "integrity", `url`). Streaming and WebSockets are deferred; the shard bridge covers live data.

store

("get", `k`, `ret`), ("put", `k`, `v`, `ack`), ("del", `k`, `ack`), ("list", `prefix`, `ret`). Keys are Str; values are any closed normal form, stored as encodings. Unforgeable names inside stored values are rejected, because a name must not outlive the tab that minted it.

```

nav
    ("go", url), ("replace", url), ("back"), ("state", v).

shard
    §9.

```

gaze-store keeps one append-only file per origin with periodic compaction. The proposal's "persistent space as a CampF1R3 checkpoint" is replaced by this key-value protocol: it needs no upstream store serialisation and is simpler to quota.

8.3 The page meter

PageMeter implements Observer. It debits a per-tab allowance for each produce, consume, COMM and aperture send, plus a per-byte charge on datum encodings, from a table in gaze-exec. The allowance is refilled per frame, larger for the foreground tab. Exhaustion aborts the operation atomically; the tab stops making progress, the shell shows why, and the user can grant more. The meter is a local budget and MUST NOT be presented as a prediction of shard cost: shard cost is computed from prost sizes of Pars (Finding 8), so deploy costs are always quoted from /api/estimate-cost.

9 The shard bridge

9.1 Components

gaze-shard runs in the browser process on tokio and depends on Embers' firefly-client for DeployData construction, secp256k1 signing, submission and the /ws/events stream. It adds the keystore, the registry resolver, the assurance ladder, the light client and sessions. Keys live in the operating system's credential store; each (user, site) pair gets its own key, generated on first grant, so sites cannot correlate a user by key.

9.2 Addresses

```

f1r3://<publisher-pubkey-hex>/<project>@<semver-range>/<path>
f1r3h://blake2b-256/<64 hex>

```

The first resolves through rho:serve:1:<pubkey>:<project>:<range>; the second names immutable content directly and may be fetched from anywhere.

9.3 Site manifest

A published version's registry entry holds, as its code, a process that sends a K1G map:

```

{ "gaze": 1, "entry": "index.html",
  "files": { "index.html": 0x9f2c..., "app.knf": 0x41aa..., "style.css": 0x07be... },
  "mirrors": [ "https://cdn.example.org/blob/" ] }

```

Every file is named by its BLAKE2b-256 hash, so the node never has to agree with the client on how to hash a Par; this settles the proposal's obligation on canonical code hashes for sites, because code travels as .knf bytes addressed by hash rather than as registry terms.

9.4 Resolution

1. Take the last finalized block from `/api/last-finalized-block` on each configured observer.
2. Query `/api/registry/{uri}?block_hash=B` for the versioned URN.
3. Verify the answer at the configured rung (§9.5).
4. Reject if B is older than the highest finalized block at which this publisher's binding was previously seen (freshness).
5. Fetch files by hash from any source (§9.7); verify each hash.
6. Subscribe: on each block-finalised event, re-resolve open sites and notify tabs of changes. When VersionedRegistry Step 7 lands, subscribe to the notify channel instead.

9.5 The assurance ladder in code

Every shard reply to a page carries its rung as the second tuple field, as in `("ok", rung, v)`, where rung is one of "node", "quorum", "proof" or "replayed".

node

one observer's answer.

quorum

identical canonical encodings from k of n configured observers run by distinct operators (default $k = 2, n = 3$).

proof

requires node work package N1: GET `/api/proof/data-at-name` and GET `/api/proof/registry/{uri}`, each returning the block header, justification signatures, `post_state_hash` and the radix-trie path. The client checks signatures against a bonded-set checkpoint shipped with the browser and advanced only through finalized blocks it has verified, requires signatures from validators holding more than two thirds of bonded stake, and recomputes the path's BLAKE2b-256 hashes up to `post_state_hash`.

replayed

fetch the process by hash and its inputs at the proof rung, and run it locally in `Replay`; available only once Obligation 1 is discharged.

9.6 Page verbs

`("lookup", uri, ret)`, `("read", name, ret)` (data at a name the site names by URN or registry entry), `("explore", knf_hash, args, ret)` (exploratory deploy), `("deploy", knf_hash, args, opts, ret)`, `("watch", uri, ch)` and `("session", uri, ret)`. `deploy` always prompts, showing the cost from `/api/estimate-cost`, and replies once with the deploy id and again with the finalization status from `/api/deploy-finalization-status`. Pages send code by hash: the bridge refuses to sign a term the page assembled from strings.

9.7 Blobs

```
pub trait BlobSource { fn get(&self, h: [u8; 32]) -> BlobFuture; fn name(&self) -> &str; }
```

Implementations, tried in order: the local content cache; mirrors named in the site manifest and in settings (GET `<mirror><hex>`); and, for blobs up to 256 KiB, the on-chain layout `F1R3Drive`

already uses — a metadata map with a first chunk inline and further chunks on sub-channels — read through data-at-name. Every source is untrusted; the hash decides.

9.8 Sessions

("session", uri, ret) makes the bridge generate a fresh key pair, deploy svc!("open", pubkey, ack) signed with it, and reply to the page with a session name — an aperture whose sends the bridge turns into deploys (or session-host messages) signed by that key. The page holds the name; the bridge holds the key. The service keys its session channel by the public key and accepts a message only from a deploy whose authenticated deployer (rho:deploy:data, rho:system:deployerId:ops, as VersionedRegistry already does) matches. DeployData's timestamp, validAfterBlockNumber and expiration_timestamp stop replay. Closing the tab destroys the key.

Interactive traffic goes to a session host: an f1r3x-serve beside a node, attached with U7's authenticated handshake, running Deterministic, which deploys its log hash to the shard every N rounds or T seconds, whichever first.

10 The graded resolver

gaze-graded implements Resolver and the weigh packing for four semirings. All arithmetic is on integers.

Semiring	Representation	$\oplus, \otimes$	Resolver
Boolean	bool	or, and	first enabled (today's behaviour)
Viterbi	Q32.32 in u64, [0, 1]	max, product rounded down	argmax; ties by candidate order
tropical	i64, $+\infty$ sentinel	min, saturating +	argmin; ties by candidate order
$\mathbb{R}_{\geq 0}$	Q32.32 in u64	saturating +, product	sample $\propto$ weight with a 128-bit draw from a xoshiro256** stream seeded from the minter seed

In a graded page, a guard is a clause: a Boolean subexpression is crisp (1 or 0), $w(n, d)$ is the scalar n/d rounded down to Q32.32, $\text{cost}(n)$ is a tropical scalar, and and and or are $\otimes$ and $\oplus$ — the fuzzyware papers' reading of conjunction and disjunction. With "fair": true the resolver adds the least positive value ε to every enabled candidate before choosing, which is the $\psi \oplus \varepsilon$ weak-fairness combinator.

Requirement 6 (Ceiling). The manifest's ceiling bounds the strength of every clause the tab may run: crisp admits Boolean clauses only, idempotent adds Viterbi and tropical, sampled adds $\mathbb{R}_{\geq 0}$. The static check covers the body at load; PageMeter::observe_consume checks every continuation at the cut, which covers code that arrived by reflection and was dropped, and aborts one that exceeds the ceiling.

Learning in the fibre, and $V = \mathbb{C}$, are out of scope; the resolver seam is where they would attach.

11 Process model

P1–P3: one process, a worker per tab.

The shell's main thread owns windows and every tab's RhoDocument. Each tab's TabExec runs

on its own worker thread. Events go from `RhoEventHandler` to the worker over a channel; commit batches come back and are applied in `RhoDocument::poll`, which then requests a re-draw. Synchronous decide drains block the main thread for at most the sync budget. Isolation between pages is by capability discipline and safe Rust.

P4: page processes.

A page process owns its `BaseDocument`, layout and painting, and ships frames to the browser process as `anyrender_serialize` scenes over shared memory; the browser process composites and forwards input. The measurement that decides between scene transport and shared GPU textures is Obligation 3. Process per site, as the proposal requires.

12 Devtools, reach, legacy and gateway

Devtools. A `rho:gaze:debug` capability, granted only to the built-in devtools page, attaches to a target tab through `f1r3x-host`'s session vocabulary (pause, step, play, scene, log) plus checkpoints and revert. Time travel keeps a ring of soft checkpoints every 60 frames (default 32 checkpoints); stepping back reverts to the nearest checkpoint and replays the log to the target frame. The devtools page is written in `f1r3lang`, which makes it the first real application of the platform. A Rhobots headset can attach to a tab the same way.

Reach tier. `gaze-reach` compiles the executive, `gaze-exec` and `gaze-dom-core` into one `cdylib` with the same export conventions as `f1r3x-wasm` (no `wasm-bindgen`). `gaze-reach.js`, a few hundred lines, supplies the `DomBackend` imports over a table of host elements indexed by `u32`, the frame callback from `requestAnimationFrame`, fetch with hash verification for net, and `IndexedDB` for store. Sessions and deploys are unavailable in the reach tier in v1, since a stock browser offers no key custody the page cannot reach.

Legacy tab (P4). `gaze-legacy` embeds the servo crate at its current LTS branch in a separate process, connected to the browser process by one channel carrying ground values encoded as K1G. It gets no grants. CEF behind the same channel is the fallback.

Gateway (P4). `gaze-gateway` runs `RhoDocument` without a window, runs the page for a bounded number of frames or to quiescence, serialises the document, and responds with ETag set to the hash, Content-Digest using sha-256 (RFC 9530 registers SHA-2, not BLAKE2b) plus `F1R3-Digest: blake2b-256=...`, an RFC 9421 signature by the gateway's own Ed25519 key (`secp256k1` is not a registered RFC 9421 algorithm) plus the publisher's `secp256k1` signature over the hash in `F1R3-Publisher-Signature`, and `Link: <f1r3://...>; rel="alternate"`.

13 Testing and conformance

Protocol conformance

a corpus of `.knf` pages with scripted injections and expected committed-DOM hashes per frame, run against `gaze-dom-blitz` and against `gaze-reach` in headless Chromium and Firefox. Both bridges MUST pass the same corpus.

Determinism

every corpus run is replayed from its log on x86_64, aarch64 and wasm32; per-frame batch-hash sequences must be identical.

Differential

kindling-eval against rho-pure-eval on generated guards over the shared subset; K1G encoding order against models Score order; CampF1R3 against rspace++ on recorded logs (Obligation 1); the graded resolvers against fuzzyware/gradedsim.py on shared examples.

Hostile proxy

a test harness proxy that rewrites, reorders, truncates and replays responses between the browser and a local three-validator shard; gate G2 requires that every rewritten resource is refused and every replayed session request rejected.

Fuzzing

cargo-fuzz targets for the .knf reader, the K1G decoder, the DOM protocol engine, and setHTML fragments.

Performance

TodoMVC with 1,000 items and a 200-mutation interaction: the executive and commit portion of a frame SHOULD stay under 4 ms at p95 on an M-series laptop. Missing it triggers the compiled-backend question the bytecode note already frames.

14 Work packages and schedule

Phases and gates are the proposal's, from 5 October 2026. The upstream packages move into P0, because gate G0 — a program changing rendered text — cannot pass without ground strings and apertures. Estimates are engineer-weeks.

WP	Phase	Deliverable	Est.
U1	P0	K1G ground data: AST, parser, normaliser, encoding, matcher, printer	3
U3, U4	P0	observer-generic machine; apertures	2
B0	P0	RhoDocument spike: K1G program sets text through a minimal bridge; frame-time and wasm-size measurements	2
E1	P1	gaze-knf, loader, manifest, grounding through a stub broker	2
E2	P1	TabExec, frame loop, recorder, replay	3
D1	P1	gaze-dom-core with the full verb set; gaze-dom-blitz	4
D2	P1	events: RhoEventHandler, static options, decide drains	2
T1	P1	conformance and determinism suites; TodoMVC in f1r3lang	2
C1	P2	broker, prompts, revocation; clock, log, rand, net, store, nav	3
S1	P2	shard bridge: keystore, resolution, quorum rung, deploys, watches, blobs	4
S2	P2	sessions; U7 in f1r3x-serve; session host with log anchoring	3
N1	P2	node: proof endpoints; client light-client check	4
M1	P2	page meter; cost quotes	1
U2, U5	P3	guards and k1ndl1ng-eval; resolver seam; Graded profile	4
G1	P3	gaze-graded: four semirings, fairness, ceiling	2
V1	P3	devtools page, time travel, headset attach	3
R1	P1– P3	reach tier	4
L1	P4	legacy process (Servo; CEF fallback)	4
W1	P4	gateway	2
X1	P4	page processes and scene transport	4
K1	P4	DNS and passkey bootstrap; chrome as a f1r3lang page; installers	4

The total is about 62 engineer-weeks over 38 calendar weeks, consistent with the proposal’s three to four engineers. The schedule risk is P0: seven engineer-weeks of work in four calendar weeks requires the upstream packages to start on day one with two engineers, one of them familiar with CampF1R3.

15 Obligations and decisions

Obligation 1. Build the `campf1r3-ispace` adapter and establish whether CampF1R3 replays `rspace++` logs; the *replayed* rung depends on it.

Obligation 2. Specify and implement round-mode resolution over contention components, and state for which pages the cut-local resolver of Remark 1 is exact.

Obligation 3. Measure scene serialisation against shared GPU textures for P4, on all three desktop platforms.

Obligation 4. Extend the K1ndl1ng sort-agreement obligation to the K1G tags, and settle whether K1G’s encoding should match the node’s Expr encoding closely enough for content hashes to agree across the two.

Obligation 5. Specify the passkey-to-session-key delegation (P-256 WebAuthn assertion authorising a `secp256k1` session key), carried over from the proposal.

15.1 Proposal decisions this specification settles, subject to review

DOM granularity

per-element verbs plus structured fragments with ref markers; whole-subtree patches as quoted processes are not needed.

Blob storage

untrusted mirrors first, F1R3Drive's on-chain layout as the fallback for small blobs, a node blob endpoint not required.

Repository and name

F1R3FLY-io/f1r3gaze, working name F1R3Gaze.

15.2 Still open

Compilation of surface f1r3lang in the browser (this specification keeps it out of v1); HTML versus a DDL-defined document language; Servo versus CEF for the legacy tab; end-to-end encryption versus session-host confinement for private sessions.

A The lamp, in K1G

```
new found, sub, clicks, on, off in {
  doc!("query1", "#lamp", *found) |
  for (@("ok", *lamp) ← found) {
    lamp!("listen", "click", *clicks, {}, *sub) |
    off!(Nil) |
    for (_ ← clicks & _ ← off) { lamp!("classAdd", "lit") | on!(Nil) } |
    for (_ ← clicks & _ ← on) { lamp!("classRemove", "lit") | off!(Nil) }
  }
}
```

Manifest: imports = [("doc", "rho:gaze:doc")], semiring boolean, ceiling crisp. The two handlers are persistent continuations that each consume one click and one token; exactly one token exists, so exactly one fires per click. The reply to listen on sub is left unconsumed, which is permitted.

References

- [1] L. G. Meredith, *A Browser with f1r3lang as its Native Execution Mechanism*, v0.2, publications/f1r3gaze/f1r3-browser-proposal.tex, 28 Sep 2026.
- [2] L. G. Meredith, *CampF1R3, K1ndl1ng, Bytecode, or the normal form?*, publications/campf1r3; implementation in F1R3FLY-io/campf1r3 (README.md, HOSTING.md).
- [3] L. G. Meredith, *Graded Where-Clauses and Races Decided by the Born Rule* (v6), publications/fuzzyware.
- [4] F1R3FLY-io/f1r3node-rust: node/src/rust/web/web_api_routes.rs, node/src/rust/web/events_info.rs, casper/src/main/resources/VersionedRegistry.rho, rho-pure-eval, rholang/src/rust/interpreter/accounting/costs.rs.
- [5] F1R3FLY-io/embers: packages/firefly-client, packages/embers/src/api.

- [6] F1R3FLY-io/F1R3Drive: docs/rholang_data_storage.md, docs/system_data_flow.md.
- [7] DioxusLabs, blitz 0.3.0-beta.2: packages/blitz-dom (Document, EventHandler, DocumentMutator, query selectors), packages/blitz-traits/src/net.rs, packages/blitz-vibey-script.
- [8] R. Polli, L. Pardue, *Digest Fields*, RFC 9530, 2024.
- [9] A. Backman, J. Richer, M. Sporny, *HTTP Message Signatures*, RFC 9421, 2024.